PYTHON CHEAT SHEET · 03 OF 05

Collections: Advanced Data Types

Specialised containers from the standard library: namedtuple, deque, Counter, defaultdict, OrderedDict, ChainMap and the User* wrappers.

IMPORT	BUILDS ON	BEST FOR	AVAILABILITY
from collections import ...	**tuple, list and dict**	**Counting, grouping, queues**	**Standard library, no install**

01 The collections family at a glance

Type	What it is	Reach for it when
`namedtuple`	A tuple whose fields have names	You want light, immutable records that read well
`deque`	Double-ended queue	Fast add/remove at BOTH ends: queues, stacks, sliding windows
`Counter`	Dict subclass that counts hashable items	Frequency tables, top-N, multiset arithmetic
`defaultdict`	Dict that creates missing values automatically	Grouping and counting without `KeyError`
`OrderedDict`	Dict with order-aware methods	`move_to_end`, FIFO `popitem`, LRU caches
`ChainMap`	A single view over several dicts	Layered settings: CLI over config over defaults
`UserDict / UserList / UserString`	Wrapper base classes	Safely customising dict, list or str behaviour

02 namedtuple: tuples with names

```
>>> from collections import namedtuple
>>> Point = namedtuple("Point", ["x", "y"])            # or "x y"
>>> p = Point(3, 4)
>>> p
Point(x=3, y=4)
>>> p.x, p[1]                                          # by name or by position
(3, 4)
>>> x, y = p                                           # still unpacks like a tuple
>>> p._asdict()                                        # convert to a dict
{'x': 3, 'y': 4}
>>> p._replace(x=10)                                   # "modify" by building a new tuple
Point(x=10, y=4)
>>> Point._fields
('x', 'y')
>>> Employee = namedtuple("Employee", "name role", defaults=["Engineer"])
>>> Employee("Ravi")
Employee(name='Ravi', role='Engineer')
>>> p.x = 99
AttributeError: can't set attribute
```

03 deque: a fast two-ended queue

Appending and popping at either end is O(1). A list pays O(n) for `pop(0)` and `insert(0, x)` because every element shifts. Indexing into the middle of a deque is O(n).

```
>>> from collections import deque
>>> q = deque([1, 2, 3])
>>> q.append(4)                                  # add on the right
>>> q.appendleft(0)                              # add on the left
>>> q
deque([0, 1, 2, 3, 4])
>>> q.pop()                                      # remove from the right
4
>>> q.popleft()                                  # remove from the left
0
>>> q.rotate(1)                                  # rotate right by one step
>>> q
deque([3, 1, 2])
>>> last3 = deque(maxlen=3)                      # bounded: oldest items fall off
>>> for n in range(6):
...     last3.append(n)
>>> last3
deque([3, 4, 5], maxlen=3)
```

04 Counter: count anything hashable

```
>>> from collections import Counter
>>> c = Counter("mississippi")
>>> c
Counter({'i': 4, 's': 4, 'p': 2, 'm': 1})
>>> c.most_common(2)                             # top-N as (item, count) pairs
[('i', 4), ('s', 4)]
>>> c["s"], c["z"]                               # missing item counts as 0, no KeyError
(4, 0)
>>> c.update("ssss")                             # adds to the counts
>>> c["s"]
8
>>> Counter(a=3, b=1) + Counter(a=1, b=5)
Counter({'b': 6, 'a': 4})
>>> Counter(a=3, b=1) - Counter(a=1, b=5)        # keeps only positive counts
Counter({'a': 2})
>>> list(Counter(a=2, b=1).elements())           # expand back to repeated items
['a', 'a', 'b']
>>> sum(c.values())                              # total number of items counted
15
```

05 defaultdict: no more KeyError

Pass a **factory** (a zero-argument callable such as `list`, `int`, `set` or `lambda: "N/A"`). Reading a missing key calls the factory, stores the result and returns it.

```
>>> from collections import defaultdict
>>> by_len = defaultdict(list)
>>> for w in ["tea", "cup", "water", "pot", "plate"]:
...     by_len[len(w)].append(w)                    # group words by length
>>> dict(by_len)
{3: ['tea', 'cup', 'pot'], 5: ['water', 'plate']}
>>> hits = defaultdict(int)                        # int() gives 0
>>> for ch in "banana":
...     hits[ch] += 1                               # count letters
>>> dict(hits)
{'b': 1, 'a': 3, 'n': 2}
>>> hits["zzz"]                                    # reading creates the key!
0
>>> "zzz" in hits
True
>>> hits.get("qqq")                                # .get() never creates a key
>>> nested = defaultdict(lambda: defaultdict(int))
>>> nested["2026"]["Nov"] += 5
>>> dict(nested["2026"])
{'Nov': 5}
```

06 OrderedDict: order-aware dictionary

Plain dicts already remember insertion order (3.7+). `OrderedDict` still adds `move_to_end()`, `popitem(last=False)` and an **order-sensitive** `==`.

```
>>> from collections import OrderedDict
>>> od = OrderedDict(a=1, b=2, c=3)
>>> od.move_to_end("a")                            # "a" becomes the last item
>>> od
OrderedDict({'b': 2, 'c': 3, 'a': 1})
>>> od.move_to_end("c", last=False)                # "c" becomes the first item
>>> list(od)
['c', 'b', 'a']
>>> od.popitem(last=False)                         # pop from the FRONT (FIFO)
('c', 3)
>>> OrderedDict(a=1, b=2) == OrderedDict(b=2, a=1)     # order matters
False
>>> {"a": 1, "b": 2} == {"b": 2, "a": 1}           # plain dicts ignore order
True
```

Example: a tiny LRU (least recently used) cache

lru_cache.py

```
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity):
        self.capacity = capacity
        self.data = OrderedDict()

    def get(self, key):
        if key not in self.data:
            return None
        self.data.move_to_end(key)             # mark as most recently used
        return self.data[key]

    def put(self, key, value):
        self.data[key] = value
        self.data.move_to_end(key)
        if len(self.data) > self.capacity:
            self.data.popitem(last=False)      # evict the least recently used

cache = LRUCache(2)
cache.put("a", 1)
cache.put("b", 2)
cache.get("a")                                 # touch "a": "b" is now the oldest
cache.put("c", 3)                              # over capacity: "b" is evicted
print("Cache keys:", list(cache.data))
print("b is cached?", cache.get("b") is not None)
```

Output

```
Cache keys: ['a', 'c']
b is cached? False
```

07 ChainMap and the User* wrappers

```
>>> from collections import ChainMap
>>> defaults = {"theme": "light", "lang": "en"}
>>> user = {"theme": "dark"}
>>> cfg = ChainMap(user, defaults)                    # lookups search left to right
>>> cfg["theme"], cfg["lang"]
('dark', 'en')
>>> cfg["font"] = 12                                  # writes go to the FIRST map only
>>> user
{'theme': 'dark', 'font': 12}
>>> cfg.maps
[{'theme': 'dark', 'font': 12}, {'theme': 'light', 'lang': 'en'}]
```

Subclassing built-in `dict` or `list` can silently bypass your overrides, because methods such as `update()` skip a custom `__setitem__`. The `User*` wrappers route every operation through your code.

```
>>> from collections import UserDict
>>> class LoudDict(dict):
...     def __setitem__(self, k, v):
...         super().__setitem__(k, v.upper())
>>> class LoudUserDict(UserDict):
...     def __setitem__(self, k, v):
...         super().__setitem__(k, v.upper())
>>> a = LoudDict()
>>> a["x"] = "hi"
>>> a.update(y="yo")                                    # bypasses __setitem__
>>> a
{'x': 'HI', 'y': 'yo'}
>>> b = LoudUserDict()
>>> b["x"] = "hi"
>>> b.update(y="yo")                                    # goes through __setitem__
>>> b
{'x': 'HI', 'y': 'YO'}
```

08 Putting it together

collections_demo.py

```
from collections import Counter, defaultdict, deque, namedtuple

text = "the quick brown fox jumps over the lazy dog the end"
words = text.split()

freq = Counter(words)                               # how often does each word appear?
print("Top 2 words:", freq.most_common(2))

by_length = defaultdict(list)                       # group the distinct words by length
for word in sorted(set(words)):
    by_length[len(word)].append(word)
print("Words by length:", dict(by_length))

Word = namedtuple("Word", "text count")             # readable records
ranked = [Word(w, n) for w, n in freq.most_common(3)]
print("Ranked:", ranked)

recent = deque(words, maxlen=3)                     # keep only the last three words
print("Last three words:", list(recent))
```

Output

```
Top 2 words: [('the', 3), ('quick', 1)]
Words by length: {5: ['brown', 'jumps', 'quick'], 3: ['dog', 'end', 'fox', 'the'], 4: ['lazy',
    'over']}
Ranked: [Word(text='the', count=3), Word(text='quick', count=1), Word(text='brown', count=1)]
Last three words: ['dog', 'the', 'end']
```

09 Also worth knowing

Tool	Use it for
`dataclasses.dataclass`	Mutable record classes with type hints; `__init__`, `__repr__`, `__eq__` generated for you
`typing.NamedTuple`	`namedtuple` written as a class, with type hints and methods
`enum.Enum`	Named constants that cannot be mistyped or reassigned
`heapq`	Priority queue (min-heap) built on a plain list
`itertools`	Iterator building blocks: `chain`, `groupby`, `combinations`, `islice`
`functools.lru_cache`	One-line memoisation for pure functions
`collections.abc`	Abstract base classes (`Mapping`, `Sequence`, …) for isinstance checks and custom containers

```
>>> import heapq
>>> h = [5, 1, 4]
>>> heapq.heapify(h)                                # in-place min-heap
>>> heapq.heappush(h, 0)
>>> heapq.heappop(h)                                # always the smallest item
0
>>> from dataclasses import dataclass
>>> @dataclass
... class Item:
...     name: str
...     qty: int = 1
>>> Item("pen", 3)
Item(name='pen', qty=3)
```